

Assignment

Scenario

You've just joined a fixed-income portfolio analytics team. You receive four data extracts covering calendar year 2025 for a bond portfolio of roughly \$850M market value. Your job is to build and deploy a small web application that loads this data into a database, computes portfolio analytics, and presents them in a dashboard. You'll then use your own app to answer the questions below.

Know this up front: real-world data extracts are imperfect, and so is this one on purpose. Part of the assignment is finding the problems, deciding how to handle them, and documenting every assumption you make. There is no trick penalty: a well-reasoned assumption we happen to disagree with scores far better than an unstated one.

Data

File	Contents
security_master.csv	One row per security: id, description, issuer, sector, rating, coupon, issue and maturity dates, asset class.
holdings_monthly.csv	Month-end position snapshots for the 12 month-ends of 2025: par, book value, market value. database_date is when the row was loaded into the warehouse, a given (security, as_of_date) pair may appear more than once.
marks_monthly.csv	Month-end clean price and option-adjusted spread (OAS, bps) per security.
transactions.csv	BUY / SELL / MATURITY events during 2025, with par and trade price.

Requirements

Architecture

- 1. Database** — An AWS-managed database (RDS PostgreSQL recommended; any AWS managed database is acceptable if you can justify the choice). The application must read from the database at runtime (not from the CSVs).
- 2. Backend** — Python. Data loading and validation, plus the analytics, exposed as a clean service layer or API.
- 3. Frontend** — A Python web framework of your choice - Plotly Dash, Flask or FastAPI with templates. Our team uses Plotly Dash in-house.

Minimum application features

- **Portfolio overview** — total market value over time, month-over-month change.
- **Allocation view** — sector and rating mix over time.
- **Security drill-down** — pick any security and see its position history, price, and OAS.
- **Data-quality page** — anomalies that your loading/validation rules detect in the data. This must be computed by rules, not a hardcoded list: if we handed your app a different extract with similar problems, the page should still find them.

Deployment

- Deployed on AWS — EC2, ECS, App Runner, Elastic Beanstalk, or Lightsail; your pick, and be ready to defend it in the debrief.
- Reachable at a public URL (plain HTTP or an IP address is fine; no auth required).
- The whole setup should fit comfortably in the AWS free tier.

- No credentials of any kind in the repository, use environment variables, AWS Secrets Manager, instance roles, or similar.

Engineering

- A GitHub repository. Python only for all application code.
- README with: an architecture diagram, setup instructions, how the data load works, and an assumptions log.
- A meaningful commit history (we will read it).
- Tests where they earn their keep; at minimum, the data-cleaning rules should be tested.

Questions to answer (short report, ≤ 2 pages or a README section)

Q1. What was the total portfolio market value at each month-end? Which month had the largest absolute change versus the prior month and how much of that change was trading activity (net purchases, sales, maturities) versus market moves? Describe your decomposition method.

Q2. Compare sector and rating allocation in January versus December. What are the three largest shifts, and was each driven by trading or by the market?

Q3. One sector had a clearly bad month in 2025. Which sector, which month, what happened to its prices and its average OAS, and what did that do to the portfolio?

Q4. Among securities held for the entire year, which 10 had the worst full-year price return? Estimate the dollar market-value impact of price alone (not trading) for each.

Q5. Data quality: list every anomaly you found in the extracts how you detected it, how you handled it, and how your app surfaces it. (We planted several; finding them is part of the score.)

Bonus. Monthly weighted-average portfolio coupon and OAS. State and justify your weighting basis.

Deliverables

1. GitHub repository link.
2. Live URL of the deployed application.
3. Report (PDF or README section) with the answers above.

Ground rules

- Any Python libraries are fine. AI coding assistants are allowed, but you must be able to explain and defend every line.
- The data is synthetic. Don't over-think market realism; the data-quality problems, however, are intentional and worth your attention.
- If something is ambiguous, make a sensible assumption and write it down, that's the job.

Good luck !